234. Palindrome Linked List

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	EASY	Solved: July 3, 2026

Problem Summary

Given the head of a singly linked list, return true if it is a palindrome, false otherwise.

Output

Find the middle using slow/fast pointers, reverse the second half in-place using the three-pointer technique, then compare the first half and reversed second half node by node.

Key Insights

- Slow/fast pointer trick: when fast reaches end, slow is at the midpoint.
- For odd-length lists, slow lands on the exact middle — that node is skipped naturally since we only iterate while right != null.
- Reversing second half reuses the **same three-pointer pattern** from LC 206.
- Comparison stops when right (shorter half) runs out — handles both odd and even correctly.

Observations

- No extra array needed — O(1) space by reversing in-place.
- right != null as loop condition is key — first half may be longer by 1 (odd case), and that middle node is safely ignored.
- This modifies the original list; can restore by reversing second half again if needed.

Points to Remember

1. **Slow/fast mid-finding pattern**: reuse for "find middle", "detect cycle", "find nth from end".
2. fast != null && fast.next != null is the standard safe condition — prevents null pointer on fast.next.next.
3. **Chaining patterns**: mid-finding → reversal → comparison is a common linked list combo (also used in reorder list LC 143).
4. Comparison loop uses right != null not left != null — always iterate the shorter/equal half to avoid going out of bounds.

Algorithm

1. Initialize slow = head, fast = head.
2. While fast != null && fast.next != null: move slow = slow.next, fast = fast.next.next.
3. slow is now the start of the second half — reverse from slow using prev/curr/next technique.
4. Set left = head, right = prev (head of reversed second half).
5. While right != null: if left.val != right.val return false, else advance both.
6. Return true.

Time Complexity & Space Complexity

O(n) — one pass for mid, one for reverse, one for compare.

O(1) — only pointers, no auxiliary storage.

Linked List

Description | Editorial | Solutions | Submissions

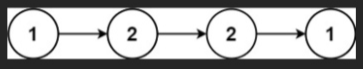
234. Palindrome Linked List

Solved

Easy | Topics | Companies

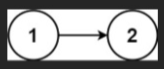
Given the `head` of a singly linked list, return `true` *if it is a palindrome or* `false` *otherwise.*

Example 1:



Input: head = [1,2,2,1]
Output: true

Example 2:



Input: head = [1,2]
Output: false

Constraints:

- The number of nodes in the list is in the range `[1, 10^5]`.
- `0 <= Node.val <= 9`

Follow up: Could you do it in `O(n)` time and `O(1)` space?

18.5K | 423 | 124 Online

Submit | Premium

Accepted

All Submissions

Accepted 93 / 93 testcases passed

Sahil Khan submitted at Jul 03, 2026 17:51

Runtime

3 ms | Beats 99.84%

Memory

94.34 MB | Beats 67.27%

Code | Java

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val =
```

View more

Write your notes here

Code

Java | Auto

```
class Solution {
    public boolean isPalindrome(ListNode head) {

        //mid mid element
        ListNode slow = head;
        ListNode fast = head;
        while(fast!=null && fast.next!=null){
            slow = slow.next;
            fast = fast.next.next;
        }

        //reverse the second half
        ListNode prev = null;
        ListNode curr = slow;
        ListNode next;
        while(curr != null){
            next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        //compare for palindrome
        ListNode left = head;
        ListNode right = prev;
        while(right!=null){
            if(left.val!=right.val) return false;
            left = left.next;
            right = right.next;
        }

        return true;
    }
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2